Java Database Connectivity

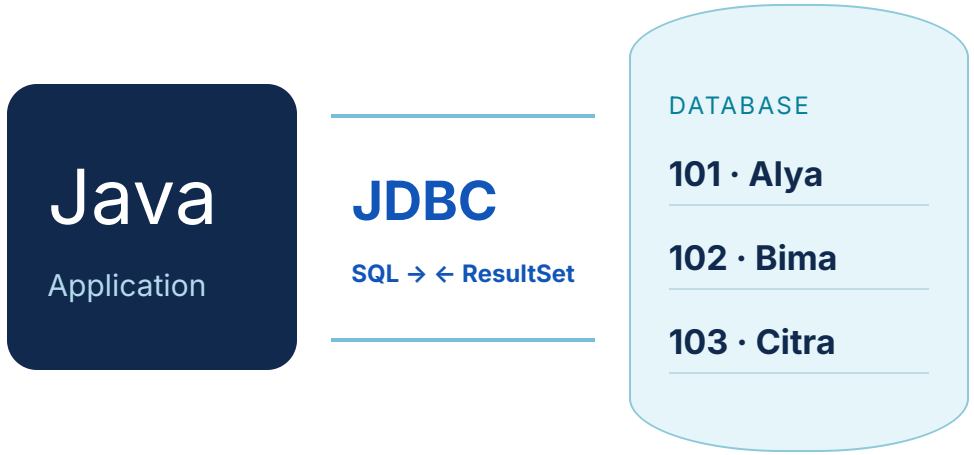
Bab 4 · Menghubungkan Java dengan data

Bagaimana sebuah baris kode menemukan satu baris data?

Arsitektur | SQL & Java | Praktik interaktif

Mulai simulasi akses JDBC →

← → untuk slide · O daftar · simulasi pada slide 11 & 20



Kompetensi pembelajaran

Setelah perkuliahan, mahasiswa mampu...

Menjelaskan

Membedakan JDBC API, driver, dan DBMS dalam satu alur komunikasi.

Membandingkan

Mengidentifikasi two-tier, three-tier, dan empat tipe driver.

Menerapkan

Membaca data dengan Connection, PreparedStatement, dan ResultSet.

Mengevaluasi

Menentukan penanganan transaksi, error, dan penutupan resource.

Saat pengguna menekan “Cari”

Kasus pengantar · Sistem informasi pegawai

1

Input

Pengguna memilih ID 101

→

2

Java

Aplikasi menyiapkan SQL

→

3

Database

DBMS mencari baris

→

4

Output

Nama ditampilkan

Di bagian mana Java berkomunikasi dengan database?

▼ **Tampilkan jawaban**

Melalui JDBC API dan driver yang sesuai dengan DBMS.

Peta konsep Bab 4

Ikuti jalur konsep sebelum menelusuri kode

JDBC

Arsitektur	Application · API · DriverManager · Driver
Model akses	Two-tier · Three-tier
Driver	Type 1 · Type 2 · Type 3 · Type 4
Workflow	Koneksi → statement → hasil → tutup
Evaluasi	Keunggulan · keterbatasan · praktik

Apa itu JDBC?

Java Database Connectivity

API standar Java untuk mengakses dan mengolah data.

Aplikasi mengirim SQL; driver menangani komunikasi dengan sumber data.

SELECT	Membaca
INSERT	Menambah
UPDATE	Mengubah
DELETE	Menghapus

JDBC adalah API konektivitas. DBMS tetap menjalankan operasi terhadap data.

Arsitektur JDBC

Satu kontrak API, implementasi komunikasi sesuai database

1

Application

Logika Java

→

2

JDBC API

Kontrak standar

→

3

Driver

Protokol DBMS

→

4

Database

SQL dan data

API

Menentukan operasi yang dapat dipanggil program.

DriverManager

Membuka koneksi melalui driver yang sesuai.

Komponen dan tanggung jawab

Klik peran untuk melihat batas pekerjaannya

Application | JDBC API | DriverManager | Driver

Application

Menentukan data yang dibutuhkan, memanggil API, dan memakai hasil.

Driver JDBC berada di runtime Java yang mengakses DBMS. Dalam three-tier, runtime itu dapat berada di application server.

Model two-tier

Aplikasi Java mengakses database secara langsung

1

Client Java

UI + logika aplikasi

→

2

JDBC driver

Di runtime client

→

3

Database server

Menerima SQL

Dua tier:
client dan database.

Driver adalah komponen komunikasi, bukan tier aplikasi terpisah.

Contoh tambahan: aplikasi desktop internal untuk pencarian pegawai.

Model three-tier

Permintaan pengguna melewati layanan aplikasi

1

Client

Antarmuka pengguna

→

2

Application server

Layanan + JDBC driver

→

3

Database server

Penyimpanan data

Client menghubungi layanan. Layanan menjalankan akses JDBC ke database.

▼ **Diskusikan: di mana validasi aturan bisnis ditempatkan?**

Application server dapat memusatkan aturan bisnis sebelum operasi database dilakukan.

Empat tipe JDBC driver

Pilih tipe untuk melihat jalur komunikasi

Type 1 | Type 2 | Type 3 | **Type 4**

Type 4 · Java penuh, protokol langsung

Java → **Thin driver** → **Database**

Type 4 banyak digunakan. JDBC–ODBC bridge adalah pendekatan lama yang tidak didukung pada Java modern.

Akses JDBC, langkah demi langkah

Jalankan satu instruksi, amati kode, resource, dan hasilnya

Java Instruksi aplikasi → **JDBC + driver** API dan protokol → **Database** Query dan hasil

```
01  // Driver PostgreSQL tersedia di classpath
02  try (Connection con = DriverManager.getConnection(url, user, password);
03       PreparedStatement ps = con.prepareStatement(
04         "SELECT name FROM employee WHERE id = ?")) {
05    ps.setInt(1, id);
06    try (ResultSet rs = ps.executeQuery()) {
07      while (rs.next()) {
08        System.out.println(rs.getString("name"));
09      }
10    } // ResultSet ditutup
11  } catch (SQLException ex) { /* tangani error */ }
```

Langkah 0 / 10

Connection	Statement	ResultSet
Belum dibuat	**Belum dibuat**	**Belum dibuat**

Siap menelusuri

Pilih skenario dan ID. Setiap klik menjalankan satu tahap akses data.

Data latihan employee

ID	NAME
101	Alya
102	Bima
103	Citra

Console `Belum ada keluaran.`

Urutan: driver → Connection → PreparedStatement → setInt() → executeQuery() → next() → getString() → akhir iterasi → tutup ResultSet → tutup statement dan koneksi. ID 101 menghasilkan Alya; ID 999 menghasilkan ResultSet kosong.

Praktik nyata: dari proyek ke koneksi

Ikuti enam tahap di komputer sendiri setelah mencoba simulasi

Unduh kode Java Unduh setup.sql Unduh driver resmi ↗

Opsi connection string (server/cloud)
DB_URL = jdbc:postgresql://host:5432/database?user=USER&password=PASSWORD&sslmode=require
Java: DriverManager.getConnection(System.getenv("DB_URL")); · Gunakan CariPegawaiConnectionString.java.
psql memakai URI postgresql://user:password@host/database, bukan JDBC URL. Ikuti SSL layanan dan encode karakter khusus.

1. Periksa JDK dan PostgreSQL

Jalankan perintah pada terminal. Pastikan javac tersedia dan PostgreSQL sudah aktif. Gunakan database latihan lokal.

```
java -version
javac -version
psql --version
```

2. Pasang JDBC driver

Unduh pgJDBC yang kompatibel dengan JDK Anda. Letakkan berkas postgresql-<versi>.jar di folder lib. Driver harus ikut classpath saat program dijalankan.

```
latihan-jdbc/
  CariPegawai.java
  setup.sql
  lib/
    postgresql-<versi>.jar
```

3. Siapkan tabel latihan

Dari folder latihan, buat database kampus memakai akun PostgreSQL dengan izin CREATEDB, lalu jalankan setup.sql melalui psql. Jika kampus sudah tersedia, lewati createdb.

```
createdb -h localhost -U nama_akun kampus
psql -h localhost -U nama_akun -d kampus -f setup.sql

# Periksa data latihan:
psql -h localhost -U nama_akun -d kampus \
  -c "SELECT id, name FROM employee;"
```

4. Periksa kode Java

Simpan CariPegawai.java pada folder latihan. URL menunjuk PostgreSQL lokal port 5432, database kampus. Akun aplikasi harus memiliki izin SELECT pada employee.

```
String url = "jdbc:postgresql://localhost:5432/kampus";

String user = System.getenv("DB_USER");
String password = System.getenv("DB_PASSWORD");
```

5. Atur kredensial runtime

Pada terminal macOS/Linux yang sama, isi nama akun database dan masukkan password secara tersembunyi. Nilai ini kredensial database latihan, bukan akun server.

```
export DB_USER="nama_akun_database"
printf "Password database: "
read -r -s DB_PASSWORD
printf "\n"
export DB_PASSWORD
```

6. Kompilasi dan jalankan

Dari folder latihan, kompilasi kode lalu jalankan dengan driver pada classpath. Nama class harus sama dengan nama file Java.

```
javac CariPegawai.java
java -cp ".:lib/*" CariPegawai

# Windows: gunakan pemisah classpath ;
java -cp ".;lib/*" CariPegawai
```

Memilih statement

Satu kebutuhan, satu objek yang sesuai

INTERFACE	KEBUTUHAN	ILUSTRASI
Statement	SQL statis	`SELECT name FROM employee`
PreparedStatement	SQL berparameter	`WHERE id = ?`
CallableStatement	Stored procedure	`{call nama_prosedur(?)}`

PreparedStatement menerima nilai parameter secara terpisah dari struktur SQL.

Parameter pada PreparedStatement

Pelengkap praktis · Pisahkan instruksi dan nilai

```
PreparedStatement ps = con.prepareStatement(
    "SELECT name FROM employee WHERE id = ?");

ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();
```

? parameter ke-1 **101**

`setInt(1, 101)`: angka pertama adalah posisi; angka kedua adalah nilai.

Binding parameter membantu mencegah SQL injection pada nilai masukan.

Placeholder mewakili nilai; bukan nama tabel atau nama kolom.

Menjalankan SQL

Bedakan bentuk hasil yang dikembalikan

executeQuery()

Untuk query yang menghasilkan ResultSet, misalnya SELECT.

executeUpdate()

Untuk INSERT, UPDATE, atau DELETE; menghasilkan jumlah baris terdampak.

```
try (PreparedStatement ps = con.prepareStatement(
        "UPDATE employee SET name = ? WHERE id = ?")) {
    ps.setString(1, "Alya Putri");
    ps.setInt(2, 101);
    int affected = ps.executeUpdate();
}
```

Contoh tambahan · affected dapat bernilai 0 jika tidak ada baris yang cocok.

Membaca ResultSet

Pelengkap praktis · Cursor dimulai sebelum baris pertama

Jalankan rs.next() | Ulangi

ID	NAME
101	Alya
102	Bima
103	Citra

Cursor: sebelum baris pertama.

```
while (rs.next()) {
    String name = rs.getString("name");
    System.out.println(name);
}
```

next() bernilai false ketika tidak ada baris berikutnya.

Data ilustratif; simulasi cursor JavaScript, bukan koneksi database.

Transaksi: commit atau rollback

Pelengkap praktis · Perubahan sebagai satu kesatuan

Auto-commit aktif secara default. Untuk satu unit kerja dengan beberapa perintah, nonaktifkan auto-commit.

```
con.setAutoCommit(false);
try {
    // jalankan operasi database
    con.commit();
} catch (SQLException ex) {
    con.rollback();
    throw ex;
}
```

Simulasi perubahan nama

Nilai tersimpan: **Alya**

Perubahan tertunda: —

UPDATE | Commit | Rollback | Ulangi

Mulai dengan UPDATE.

Model sederhana satu transaksi; tanpa konkurensi.

Resource dan SQLException

Pelengkap praktis · Dua tanggung jawab berbeda

Tutup resource

Gunakan try-with-resources untuk Connection, Statement, dan ResultSet.

ResultSet → Statement → Connection

Resource ditutup saat keluar dari blok, termasuk ketika terjadi exception.

Telusuri error

```
catch (SQLException ex) {
    System.err.println(ex.getMessage());
    System.err.println(ex.getSQLState());
    System.err.println(ex.getErrorCode());
}
```

Untuk diagnosis lokal. Pada aplikasi produksi, gunakan logging terkontrol dan pesan pengguna yang aman.

Penutupan resource tidak menggantikan keputusan commit atau rollback.

Contoh utuh: mencari pegawai

Adaptasi contoh artikel · ID 101 → nama pegawai

```
import java.sql.*;

public class CariPegawai {
  public static void main(String[] args) {
    String url = "jdbc:postgresql://localhost:5432/kampus";
    try (Connection con = DriverManager.getConnection(url,
           System.getenv("DB_USER"), System.getenv("DB_PASSWORD"));
         PreparedStatement ps = con.prepareStatement(
           "SELECT name FROM employee WHERE id = ?")) {
      ps.setInt(1, 101);
      try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) System.out.println(rs.getString("name"));
      }
    } catch (SQLException ex) {
      System.err.println("Akses database gagal: " + ex.getSQLState());
    }
  }
}
```

Prasyarat: tabel employee, driver PostgreSQL, dan kredensial environment. Telusuri kode langkah demi langkah →

Praktikum: uji jalur akses data

Coba ID 102, ID 999, lalu skenario gagal; bandingkan setiap tahap

Java Instruksi aplikasi → **JDBC + driver** API dan protokol → **Database** Query dan hasil

```
01  // Driver PostgreSQL tersedia di classpath
02  try (Connection con = DriverManager.getConnection(url, user, password);
03       PreparedStatement ps = con.prepareStatement(
04         "SELECT name FROM employee WHERE id = ?")) {
05    ps.setInt(1, id);
06    try (ResultSet rs = ps.executeQuery()) {
07      while (rs.next()) {
08        System.out.println(rs.getString("name"));
09      }
10    } // ResultSet ditutup
11  } catch (SQLException ex) { /* tangani error */ }
```

Langkah 0 / 10

Connection	Statement	ResultSet
Belum dibuat	**Belum dibuat**	**Belum dibuat**

Siap menelusuri

Pilih skenario dan ID. Setiap klik menjalankan satu tahap akses data.

Data latihan employee

ID	NAME
101	Alya
102	Bima
103	Citra

Console `Belum ada keluaran.`

Urutan: driver → Connection → PreparedStatement → setInt() → executeQuery() → next() → getString() → akhir iterasi → tutup ResultSet → tutup statement dan koneksi. ID 101 menghasilkan Alya; ID 999 menghasilkan ResultSet kosong.

Model Java dan tabel PostgreSQL

Pelengkap praktik · Objek adalah representasi data pada saat dibaca

Model Java: Employee

```
public class Employee {
  private final int id;
  private String name;
  public Employee(int id, String name) {
    this.id = id; this.name = name;
  }
  public int getId() { return id; }
  public String getName() { return name; }
  public void setName(String name) {
    this.name = name;
  }
}
```

Mapping baris → objek

```
// SELECT id, name FROM employee WHERE id = ?
if (rs.next()) {
  Employee employee = new Employee(
      rs.getInt("id"), rs.getString("name"));
}
```

`id INTEGER` → `int id`

`name VARCHAR` → `String name`

ResultSet dibaca dan nilainya disalin ke objek. Perubahan field objek perlu disimpan dengan SQL.

JDBC membutuhkan mapping eksplisit. Contoh ini tidak memakai ORM.

Sinkronisasi model dan database

Ubah model, simpan ke DB, lalu baca ulang nilai yang tersimpan

Model di memori Java

ID 101

Alya

Nilai sama

UPDATE →

← SELECT

Tabel employee · PostgreSQL

ID 101

Alya

```
Employee e = repository.findById(con, 101);
```

Setelah DB berubah, tekan Baca ulang SELECT untuk memperbarui objek.

Unduh repository · Unduh model

Awal: model dan database memuat Alya. Coba ubah model tanpa menyimpan.

Model tidak otomatis tersinkron. Simulasi satu baris memakai auto-commit; tanpa kontrol konflik antar-pengguna.

Ringkasan kelas dan interface

Kenali objek, peran, dan jenisnya

NAMA	JENIS	PERAN
DriverManager	Class	Membuka koneksi melalui driver
Types	Class	Konstanta tipe SQL
Driver · Connection	Interface	Implementasi driver dan sesi database
Statement · PreparedStatement	Interface	Perintah SQL statis / berparameter
CallableStatement	Interface	Pemanggilan stored procedure
ResultSet · RowSet	Interface	Hasil query / pengolahan data tabular
Blob · Clob	Interface	Data biner besar / karakter besar
SQLException	Class	Informasi kegagalan akses SQL

Keunggulan dan keterbatasan

Tentukan kecocokan JDBC dengan kebutuhan aplikasi

Keunggulan

- API umum untuk beberapa DBMS.
- Akses SQL secara langsung.
- Parameter, transaksi, dan ResultSet.
- Lintas platform dengan driver kompatibel.

Keterbatasan

- Driver harus sesuai target database.
- SQL dan resource dikelola pengembang.
- SQLException perlu ditangani.
- Aplikasi besar memerlukan lapisan tambahan.

Pelengkap: DataSource lazim dipilih untuk pengelolaan koneksi. Pooling, pemetaan objek, dan abstraksi seperti JPA atau Spring Data berada di atas akses dasar JDBC.

Knowledge check

Pilih jawaban; gunakan umpan balik untuk memperbaiki pemahaman

1. Dalam three-tier, komponen mana yang menjalankan JDBC?

A. Browser · B. Application server
C. Database server · D. Semua tier

2. Apa arti setInt(1, 101)?

A. Kolom 101 diisi angka 1
B. Parameter ke-1 diberi nilai 101

3. Apa hasil executeUpdate() untuk UPDATE?

A. ResultSet · B. Nama tabel
C. Jumlah baris terdampak · D. Boolean

4. Sebelum next() pertama, di mana cursor berada?

A. Baris pertama · B. Baris terakhir
C. Setelah semua baris · D. Sebelum baris pertama

5. Setter pada model Java langsung mengubah database?

A. Model Java saja yang berubah · B. Database ikut berubah

6. Bagaimana memperbarui model setelah data di DB berubah?

A. Otomatis · B. Tutup koneksi · C. SELECT dan mapping ulang

Kunci dosen: 1 B · 2 B · 3 C · 4 D · 5 A · 6 C

Diskusi dan refleksi

Gunakan alasan teknis, bukan hanya nama API

01 Portal pegawai diakses melalui browser.
Di tier mana JDBC sebaiknya bekerja?

02 UPDATE berhasil, tetapi proses berikutnya gagal.
Kapan rollback diperlukan?

03 Database dipindah dari PostgreSQL lokal ke server lain.
Apa yang perlu diperiksa selain driver?

▼ **Panduan diskusi dosen**

Application server; satu transaksi eksplisit untuk operasi terkait; periksa URL, kredensial, skema, dialek SQL, dan dukungan fitur driver.

Java mengirim instruksi.
Database mengolah data.

JDBC menghubungkan keduanya melalui driver yang sesuai.

Hubungkan

API standar + driver sesuai DBMS.

Jalankan

Connection → PreparedStatement → ResultSet.

Kelola

Transaksi, error, dan penutupan resource.

▼ Referensi utama & pelengkap teknis

- pgJDBC · Koneksi PostgreSQL
- GeeksforGeeks · Introduction to JDBC
- Oracle · JDBC Architecture
- Oracle · Establishing a Connection
- Oracle · PreparedStatement API
- Oracle · Using Transactions
- PostgreSQL · psql
- Oracle · java.sql API
- Oracle · Processing SQL Statements
- Oracle · Using Prepared Statements
- Oracle · ResultSet API
- Oracle · Handling SQLExceptions

Latihan lanjutan: buat pencarian ID, tangani hasil kosong, lalu jelaskan urutan penutupan resource.